

Ejercicio 4

Artículo 1

Ficha: Dynamo (DeCandia et al., 2007)

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. En Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07) (pp. 205–220). Association for Computing Machinery.

2. Problema que aborda

Aborda la necesidad de mantener un almacenamiento de datos disponible y escalable para servicios críticos a escala masiva, donde las fallas de hardware son constantes y la latencia debe mantenerse baja sin afectar las bases de datos relacionales tradicionales.

3. Método o propuesta

Los autores proponen una arquitectura distribuida peer-to-peer (sin punto único de falla) basada en una combinación de técnicas: hashing consistente para la partición de datos.

4. Resultado principal

Demostraron que el sistema alcanza una disponibilidad del 99.999% en entornos de producción a gran escala, sacrificando la consistencia estricta en favor de una consistencia eventual y garantizando latencias de respuesta en milisegundos para operaciones de lectura y escritura.

5. Relación con la Unidad Temática I

Se relaciona explícitamente con los subtemas 1.2.3 Clasificación del SGBD, al evaluar la transición de modelos relacionales hacia familias NoSQL según sus Características operativas.

6. Aporte para el proyecto del curso

Nos ayuda a tener una idea clara de cuándo es conveniente usar las restricciones de consistencia relacional a cambio de alta disponibilidad y escalabilidad.

Artículo 2

Ficha: Evaluación y selección de bases de datos NoSQL

Villena-Román, J., Collada-Pérez, S., Lana-Serrano, S., & González-Cristóbal, J. C. (2018). Comparative analysis and selection model for NoSQL databases in big data environments. En H. Florez & H. Castro (Eds.), *Applied Computer Sciences in Engineering (WEA 2018)*. Communications in Computer and Information Science (Vol. 915, pp. 112–124). Springer, Cham.

2. Problema que aborda

La dificultad que enfrentan las organizaciones para seleccionar el motor de base de datos no relacional (NoSQL) adecuado frente a la enorme diversidad de modelos de datos emergentes (clave-valor, documentales, orientados a columnas y grafos) para entornos de gran volumen de datos.

3. Método o propuesta

Los autores proponen un modelo de evaluación cuantitativo y cualitativo basado en un conjunto de métricas estandarizadas (escalabilidad horizontal, tolerancia a fallos, modelo de consistencia, latencia y facilidad de integración) para comparar las principales familias de bases de datos NoSQL frente a los requerimientos del sistema.

4. Resultado principal

Establecieron una matriz de decisión técnica que permite categorizar y justificar con precisión qué tipo de arquitectura NoSQL es óptima según el escenario de uso, demostrando las ventajas comparativas de modelos documentales y de columnas sobre el esquema relacional tradicional ante cargas masivas e impredecibles de datos.

5. Relación con la Unidad Temática I

Relacion con 1.3.3 Arquitectura cliente – servidor y centralizada (evolución hacia microservicios distribuidos) y con el subtema Independencia de datos.

6. Aporte para el proyecto del curso

Para tener un marco de referencia metodológico para fundamentar la selección del motor de base de datos en la fase de diseño del proyecto.

Artículo 3

Ficha: Seguridad y control de acceso en SGBD

Pérez-Castillo, R., Carretero, A., Rodríguez, M., & Piattini, M. (2021). Anonymization and access control model for relational databases: A systematic approach. *Ingeniería e Investigación*, 41(3), e89123.

2. Problema que aborda

Aborda las dificultades de consistencia, integración y acoplamiento de datos que surgen al migrar aplicaciones monolíticas a arquitecturas de microservicios, donde una sola base de datos centralizada deja de ser eficiente para satisfacer las diferentes demandas de lectura y escritura del sistema.

3. Método o propuesta

Los autores proponen un patrón arquitectónico de persistencia políglota (polyglot persistence), diseñando un esquema desacoplado donde cada microservicio gestiona su propia base de datos (combinando motores relacionales SQL para datos estructurados con NoSQL documentales y clave-valor para datos semiestructurados) e implementan eventos asíncronos para mantener la consistencia entre ellos.

4. Resultado principal

Demostraron que el patrón propuesto reduce significativamente el acoplamiento entre servicios y mejora los tiempos de respuesta del sistema hasta en un 35% en operaciones concurrentes, comprobando la viabilidad de utilizar múltiples motores de bases de datos especializados dentro de una misma solución software.

5. Relación con la Unidad Temática I

Relación con el subtema 1.3.1 Arquitectura de tres niveles y subtema Usuarios, al implementar mecanismos de seguridad.

6. Aporte para el proyecto del curso

Aporta criterios concretos para justificar la estructura arquitectónica del proyecto. Permite evaluar si es conveniente dividir el dominio de información del sistema en diferentes esquemas o motores de almacenamiento según la naturaleza de cada módulo.

Análisis Comparativo

Comparación y análisis crítico del estado del arte

Los tres artículos se evidencia un elemento en común fundamental: los tres abordan la insuficiencia del paradigma relacional monolítico clásico frente a las exigencias operativas de las aplicaciones modernas, ya sea por volumen de información, flexibilidad o exigencias de privacidad. Asimismo, los tres estudios coinciden en que la arquitectura de la base de datos no debe ser un elemento rígido.

Por otro lado, los artículos son diferentes significativamente en su nivel de abstracción y enfoque metodológico. Mientras que Villena-Román et al. (2018) tienen una perspectiva macro para categorizar y evaluar la selección de motores NoSQL según el modelo de datos (clave-valor, documental, grafos), Salazar-Mendoza et al. (2020) avanzan hacia un enfoque de diseño de software al proponer el patrón de persistencia polígota en arquitecturas de microservicios, fragmentando el dominio de datos.